01

MEMO Ontology

Medical Engineering Modeling Ontology

Model-driven safety traceability
for safety-critical medical devices

Open source semantic backbone
for regulated medical systems

MEMO

02 STORY

It is **11 p.m. before an audit.** An engineer changes one line in **REQ-145**. The next morning, no one in the room can answer the question that follows: **"What evidence is still valid?"**

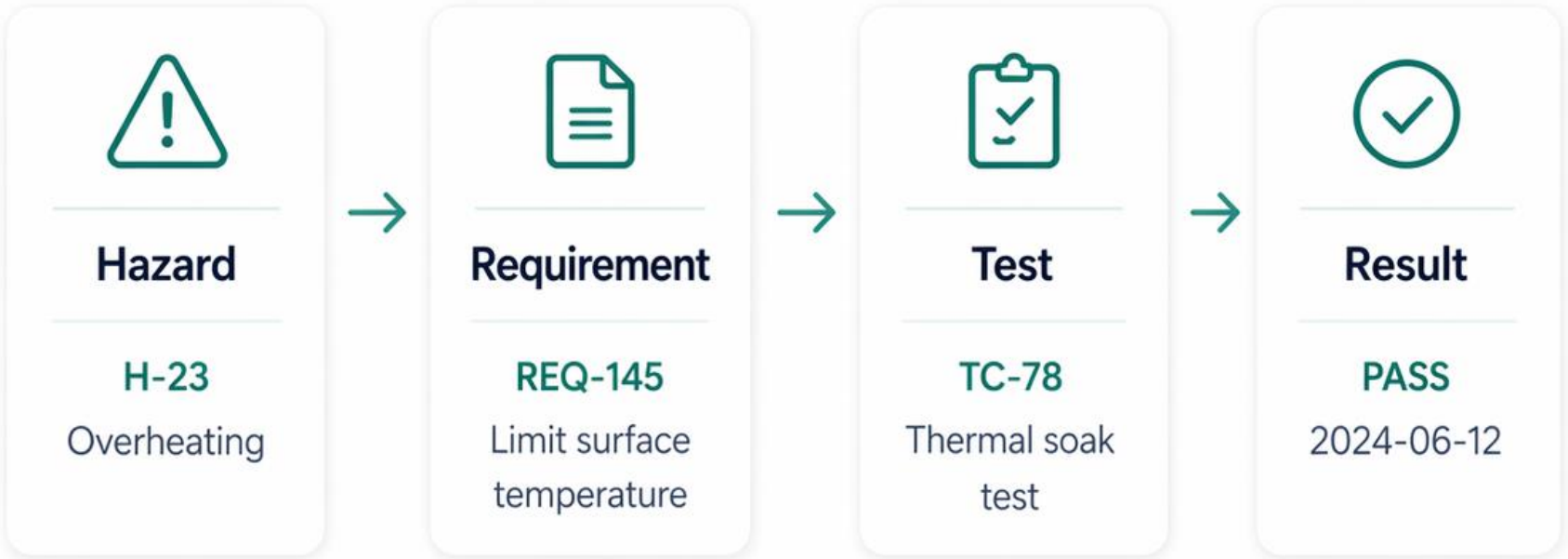
A small change. A clean diff. And a safety case that quietly stopped meaning what it said.

— A familiar story across regulated medical engineering

02 PROBLEM

Evidence is not stable under change

A small change can break the chain.



The challenge

Links exist, but their meaning is implicit. When intent changes, we cannot know what is truly impacted.

Changing REQ-145 invalidates TC-78 and its result.

03 EVIDENCE DRIFT

Evidence Drift

How traces degrade as systems evolve.

Aligned baseline

All links valid.
Evidence is sound
and defensible.

Partial mismatch

Some links stale.
Unclear what is affected
or still valid.

Broken trace

Critical links invalid.
Trace cannot support
safety claims.

Drift is expensive — and quiet

Unverified change is the silent driver of recalls, rework, and audit findings.

21%

FDA recalls

Software & traceability flagged in roughly one-fifth of Class II/III recalls (FDA MAUDE, 2023).

~40%

Rework cost

Late-stage requirement and risk rework absorbs 30–45% of typical V&V budgets.

DAYS

Audit time

Reconstructing a single broken trace before audit: days, not minutes.

↻

Lost confidence

Each silent break weakens the safety case the next reviewer reads.

The cost is not the link. It is the doubt every broken link injects into the case.

04 INDUSTRY CONTEXT

Other safety-critical domains solved this

They share a common semantic backbone.

Aerospace

ARP4754A / ARP4761

Formal safety models

Mature trace practices

Automotive

ISO 26262

Structured safety cases

Tool-integrated traces

Medical Devices

ISO 14971 / 13485

Risk & usability traces

Regulatory alignment

Different domains, same lesson: semantics enable trust.

05 KEY INSIGHT

The gap is semantic, not tooling

Tools can store links. Only a shared meaning model can ensure those links stay correct.

> Without semantics, traceability is a graph.
>
> **With semantics, it is a guarantee.**

Implicit intent

Different teams, different meaning

Ad-hoc links

Inconsistent and brittle

Manual impact

Guesswork and missed ripple

Audit risk

Harder to justify safety claims

06 OUR SOLUTION

MEMO defines the missing semantic layer

A domain ontology for safety-critical medical devices.

Typed elements

Well-defined medical-device artifacts with clear attributes.

Typed relationships

Meaningful connections that express intent and constraints.

Closure rules

Logical rules that complete the chain and enforce consistency.

Medical Engineering Modeling Ontology: purpose-built for regulated medical systems.

Built on what you already practice

MEMO is not a new methodology. It is a SysML v2 ontology underneath the methodologies your teams use.

ISO

ISO/IEC/IEEE 42010

Concerns → Viewpoints → Views → Models. md::viewpoints + md::views supply the typed elements that views reference.

MBSE

Arcadia-inspired layering

Operational → Logical → Physical → Behavior, with risk and assurance as peers — not after-thoughts.

v2

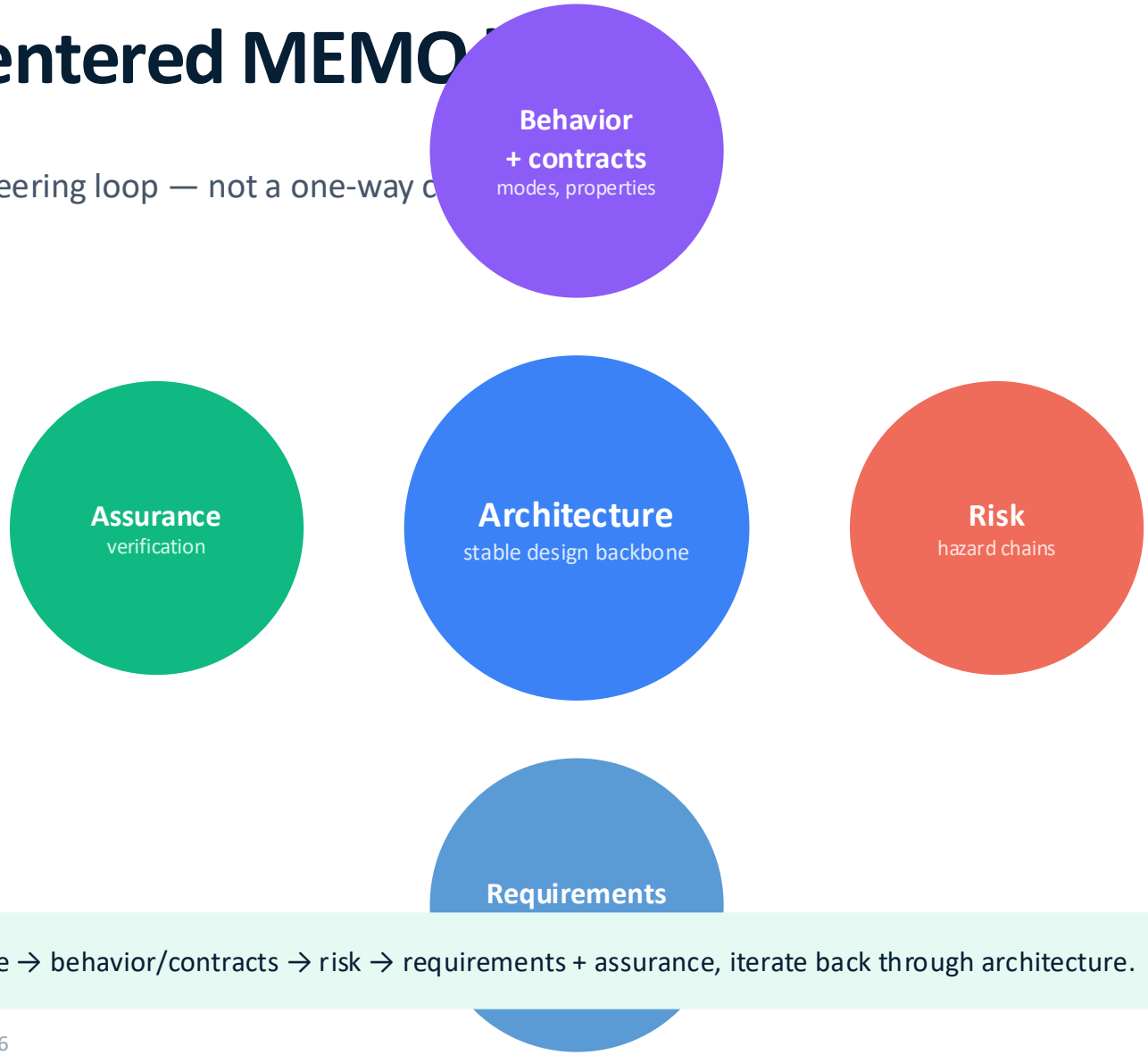
SysML v2 source

All kinds, links, viewpoints, methodology in SysML v2. Single coherent namespace md::. Single release version.

Position. MEMO sits between the standards (ISO 14971, IEC 62304, IEC 81001-5-1) and the modeling tools — a domain ontology, not a tool stack.

Architecture-centered MEMO

Use MEMO as an iterative engineering loop — not a one-way c



CONTEXT

stakeholders, care setting, intended use

VIEWS / DOCUMENTS

RMF · SDD · V&V · compiled views

Practitioner sequence. Architecture → behavior/contracts → risk → requirements + assurance, iterate back through architecture.

One namespace. One release. md::

MEMO ships as a single SysML v2 package — md::library — versioned 1.0.0. Layers are sub-packages, not separate libraries.

```
medical_device_library.sysml
  package md::library {
  public import md::core::*;                // abstract bases, enums, links
  public import md::manifest::release::*;   // version, changelog

  public import md::architecture::context::*;
  public import md::architecture::requirements::*;
  public import md::architecture::functions::*;
  public import md::architecture::behavior::*;
  public import md::architecture::logical_structure::*;
  public import md::architecture::software_structure::*;
  public import md::architecture::hardware_structure::*;
  public import md::architecture::risk::*;
  public import md::architecture::cybersecurity::*;
```

Single namespace

md:: spans every layer. No MEMO_Ontology_*. Imports are short and predictable.

Single release version

1.0.0 covers core, architecture, methodology, viewpoints, views, compliance, examples.

Layers ≠ libraries

Architecture sub-packages are layers. Viewpoints + views are first-class libraries.

Source-of-truth

Ontology stays the source of truth; methodology guides without owning viewpoints.

Seven sub-packages under md::

Each sub-package is a focused library. Architecture is decomposed into 13 ontology layers.

md::core	*abstract bases · enumerations · semantic links*	common · enumerations · relationships
md::architecture	*13 ontology layers — what the device is*	context · requirements · functions · behavior · logical_structure · logical_interfaces · software_structure · hardware_structure · physical_interfaces · constraints · risk · cybersecurity · assurance
md::methodology	*how to apply the ontology*	core · profiles · viewpoints · rules · patterns · workflow · gates
md::viewpoints	*reusable selection intent*	core · default_viewpoints
md::views	*model-driven content + presentation*	core · document_views
md::compliance	*controlled artifacts + document views*	artifacts · document_views
md::examples::gpca	*worked example — GPCA infusion pump*	architecture · behavior_modes · behavior_subsystems · interfaces · requirements · risk · cybersecurity · verification · formal_behavior · methodology · trace · views · document_views

Abstract bases, enumerations, semantic links

The whole ontology specializes a small set of bases. Enums encode categorical attributes. Links carry typed semantics.

core/md_common.sysml

```
package md::core::common {


part def IdentifiedElement {
    attribute id : String;
    attribute name : String;
    attribute description : String;
  }


part def TraceableElement :> IdentifiedElement {
    attribute rationale       : String;
    attribute sourceReference : String;
  }


part def ArchitectureElement :> TraceableElement;

part def VerifiableElement   :> TraceableElement;

part def EvidenceElement     :> TraceableElement;
```

core/md_enumerations.sysml + md_relationships.sysml

```
package md::core::enumerations {

enum def ConcernKind { safety; usability; cybersecurity;
                       performance; privacy; regulatory; }

enum def SafetyClassKind { none; A; B; C; }

enum def HazardTypeKind { overdose; underdose; occlusion;
                          freeFlow; airInLine; reverseDelivery; …
}

enum def ThreatCategoryKind { spoofing; tampering; …;
                              informationDisclosure;
elevationOfPrivilege; }
}

package md::core::relationships {

part def SemanticLink  :> TraceableElement;

part def RequirementSatisfactionLink :> SemanticLink;
```

28+ enums. ConcernKind · CriticalityKind · SafetyClassKind · HazardTypeKind · RiskControlKind · ThreatCategoryKind · CyberControlKind · AudienceKind · WorkflowStageKind · DocumentViewKind …

Thirteen ontology layers under md::architecture

Each layer is a sub-package. Layers progress from intent and need to solution and evidence; risk and cybersecurity are peers.

context	Actor, UseContext
functions	LogicalFunction, LogicalFlow, DataDefinition, ControlDefinition
logical_structure	LogicalComponent
software_structure	SoftwareSystem, SoftwareComponent (period, deadline, WCET, scheduling)
physical_interfaces	PhysicalInterface, PortBinding
risk	Hazard, RiskBeforeMitigation, RiskAfterMitigation, RiskMatrix, RiskControl
assurance	VerificationCase, TestArtifact, Evidence

requirements	StakeholderNeed, Requirement, SystemRequirement, SoftwareRequirement, HardwareRequirement
behavior	BehaviorMachine, ModeState, Transition, Contract, AssumeProperty, GuaranteeProperty
logical_interfaces	LogicalInterface (domain · kind · pattern)
hardware_structure	HardwareAssembly
constraints	ConstraintDefinition (timing, resource, env)
cybersecurity	CybersecurityAsset, AttackSurface, Threat, Vulnerability, Mitigation

Reading. Cybersecurity is a peer to safety/software/hardware/assurance — modeled as ontology layer, not bolted on later.

Stakeholders, intended use, requirements as kinds

Specialize abstract bases. Attributes carry the regulator-relevant fields.

architecture/md_context.sysml

```
package md::architecture::context {

part def Actor :> TraceableElement {
    attribute actorKind     : String;
    attribute trainingLevel : String;
  }


part def UseContext :> TraceableElement {
    attribute careSetting   : String;
    attribute environment   : String;
    attribute jurisdiction  : String;
    attribute connectedUse  : Boolean;
  }
}
```

architecture/md_requirements.sysml

```
package md::architecture::requirements {

requirement def StakeholderNeed :> RequirementDriver {
    attribute statement : String;
    attribute priority  : String;
  }


requirement def Requirement :> VerifiableElement {
    attribute statement          : String;
    attribute sourceKind         :
RequirementSourceKind;
    attribute concernKind        :
ConcernKind;
    attribute status             :
RequirementStatusKind;
    attribute acceptanceCriteria : String;
    attribute assumptionSummary  : String;
    attribute guaranteeSummary   : String;
  }
```

SysML idiom. Requirements use SysML v2 requirement def — they are first-class verifiable elements, not docstring fields on parts.

Functions, modes, contracts — assume / guarantee built in

Behavior carries formal properties. Contracts attach to architecture elements and feed model checking.

architecture/md_functions.sysml

```
package md::architecture::functions {

part def LogicalFunction :> ArchitectureElement {
    attribute functionCategory : String;
    attribute trigger          : String;
    attribute concernKind      :
ConcernKind;
    attribute criticality      :
CriticalityKind;
  }


part def LogicalFlow :> ArchitectureElement {
    attribute flowKind        :
FlowKind;
    attribute direction       :
DirectionKind;
    attribute latencyBudgetMs : Real;
    attribute integrityLevel  : String;
  }
}
```

architecture/md_behavior.sysml

```
package md::architecture::behavior {

part def BehaviorMachine :> ArchitectureElement;

part def ModeState      :> ArchitectureElement;

part def Transition     :> ArchitectureElement {
    attribute trigger         : String;
    attribute guardSummary    : String;
    attribute effectSummary   : String;
    attribute sameStepCritical: Boolean;
  }


part def BehaviorProperty :> VerifiableElement {
    attribute propertyKind     :
BehaviorPropertyKind;
    attribute languageKind     :
PropertyLanguageKind;
    attribute formalExpression : String;
  }
```

Properties carry language. AGREE-like, LTL-like, CTL-like — property language is captured, so model checkers can target the right ones.

part def GuaranteeProperty :> BehaviorProperty;

Safety and security as peer ontology layers

Hazard chain on one side, threat chain on the other — both feed the same RequirementDriver / VerifiableElement contract.

architecture/md_risk.sysml

```
package md::architecture::risk {

part def Hazard :> TraceableElement {
    attribute hazardType  :
HazardTypeKind;
    attribute foreseeable : Boolean;
  }

part def RiskBeforeMitigation :> Risk {
    attribute foreseeableMisuseIncluded : Boolean;
  }

part def RiskAfterMitigation  :> Risk {
    attribute residualAcceptability : String;
    attribute benefitRiskRequired   : Boolean;
  }

part def RiskControl :> VerifiableElement {
    attribute controlKind :
RiskControlKind;
  }
```

architecture/md_cybersecurity.sysml

```
package md::architecture::cybersecurity {

part def CybersecurityAsset :> ArchitectureElement {
    attribute assetKind            :
AssetKind;
    attribute confidentialityNeed  : String;
    attribute integrityNeed        : String;
    attribute availabilityNeed     : String;
    attribute safetyRelevant       : Boolean;
  }


part def AttackSurface :> TraceableElement {
    attribute entryPointKind          :
InterfaceKind;
    attribute exposureLevel           : String;
    attribute authenticationExpected  : Boolean;
  }


part def Threat :> RequirementDriver {
```

Why peers. Threat :> RequirementDriver — exactly like Risk. Same downstream machinery generates security requirements and verification.

Links carry roles — not just direction

Every link is a SemanticLink subtype with named part roles. Linker checks kind on each end.

core/md_relationships.sysml

```
package md::core::relationships {

part def SemanticLink :> TraceableElement {
    attribute linkStatus :
LinkStatusKind;
  }


part def RequirementSourceLink :> SemanticLink {
    attribute sourceRole       : String;
    part      sourceDriver     :
RequirementDriver;
    part      targetRequirement:
VerifiableElement;
  }


part def FunctionAllocationLink :> SemanticLink {
    part function          :
ArchitectureElement;
    part allocatedElement  :
ArchitectureElement;
}
```

Requirement	RequirementSourceLink · RequirementSatisfactionLink
Function	FunctionAllocationLink
Interface	InterfaceRealizationLink · TrustBoundaryCrossingLink
Verification	VerificationLink · EvidenceProductionLink · DocumentInclusionLink
Risk	RiskTraceLink (initiates · leadsTo · canCause · resultsIn · aggregates …)
Cyber	AssetThreatLink · ThreatVulnerabilityLink · CyberSafetyTraceLink
Execution	ExecutionOrderLink (sameStepRequired)
Methodology	MethodologyBindingLink

Methodology shapes how the ontology is applied

Definitions select rigor, viewpoints, rules, patterns, gates. Resolved methodologies bind to a project.

methodology/md_profiles.sysml

```
package md::methodology::profiles {


part mdCoreLibrary : MethodologyLibrary {
    attribute id      =
"METHLIB-001";
    attribute name    =
"MedicalDeviceMethodologyLibrary";
    attribute version =
"0.1";
  }


part mdLightDefaultDefinition : MethodologyDefinition {
    attribute description =
"Light default methodology around the
     ontology for smaller or early project use."
;
  }


part mdLightDefaultResolved : ResolvedMethodology {
    attribute rigor =
MethodRigorKind::lightweight;
  }
```

core	MethodologyDefinition · ResolvedMethodology · ProjectMethodBinding
profiles	instances: light, balanced, formal
viewpoints	default viewpoints per audience/stage
rules	ViewRule (required · recommended · forbidden)
patterns	ModelingPattern — recurring shapes
workflow	WorkflowStage sequencing
gates	ReviewGate, exit criteria

Viewpoints declare intent. Views bind it to content.

First-class libraries — not buried under methodology or compliance. Selection queries are explicit.

viewpoints/md_viewpoint_core.sysml

```
package md::viewpoints::core {


part def Viewpoint :> DocumentedElement {
    attribute purpose                  : String;
    attribute audience                 :
AudienceKind[*];
    attribute stage                    :
WorkflowStageKind;
    attribute outputKind               :
ViewOutputKind[*];
    attribute presentationKind         :
PresentationKind[*];
    attribute concern                  :
ConcernKind[*];
    attribute includedLayers           : String[*];
    attribute allowedElementKinds      : String[*];
    attribute allowedRelationshipKinds : String[*];
    attribute filterExpression         : String;
  }
}
```

views/md_view_core.sysml

```
package md::views::core {


part def ViewSelectionQuery :> TraceableElement {
    attribute includeElementKinds      : String[*];
    attribute includeRelationshipKinds : String[*];
    attribute includeLayers            : String[*];
    attribute includeConcerns          :
ConcernKind[*];
  }


part def View :> DocumentedElement {
    part viewpoint            :
Viewpoint;
    ref  exposesElement       :
TraceableElement[*];
    ref  exposesRelationship  :
SemanticLink[*];
    part selectionQuery       : ViewSelectionQuery[*];
  }
```

Separation. Viewpoint = intent (audience, stage, allowed kinds). View = bound content (selection query, exposed elements, presentation).

part def DocumentBackedView :> View {

Extend through methodology profiles

Pick rigor. Pick viewpoints. Pick rules. Bind to project. The ontology stays unchanged.

1 Pick rigor

`MethodRigorKind`

```
rigor = lightweight
      | balanced
      | formal
```

2 Pick viewpoints

`Viewpoint instances`

```
audience = safetyEngineer
stage    = risk
```

3 Author rules

`ViewRule instances`

```
elementTypeName = "Hazard"
relationTypeName = "RiskTrace"
strength = required
```

4 Bind to project

`ProjectMethodBinding`

```
projectName = "GPCA Pump"
resolvedMethodology = ...
```

5 Add domain kinds

`your own md::* package`

```
part def InfusionMode :>
     ModeState { ... }
```

Rule. Add packages and methodology profiles. Don't edit core. Methodology guides without owning viewpoints.

Meet GPCA — md::examples::gpca

Generic Patient-Controlled Analgesia infusion pump. Public-domain U. Minnesota CriSys benchmark, IEC 62304 Class C.

WHY GPCA

- **U. Minnesota CriSys benchmark**
- Public REQ catalogue (REQ 1–86)
- IEC 62304 Class C software
- Full ISO 14971 hazard chain published
- Cited across FDA, NIH, INCOSE literature

WHAT WE MODELED

- Stakeholders, intended use, user needs
- Hierarchical mode machine OFF/IDLE/PAUSED/BASAL/SQUARE/PATIENT_BOLUS
- Software components TLM · Alarm · InfusionMgr · 5+ more
- Hazard chain + lockout RiskControl + benefit-risk eval
- AGREE-like guarantees + LTL temporal properties

md::examples::gpca::*

requirements	needs · system & software requirements
architecture	hardware assemblies + software components
interfaces	logical + physical interfaces
behavior_modes	16-state hierarchical mode machine
behavior_subsystems	function decomposition per subsystem
formal_behavior	Contracts · AGREE-like · LTL properties
risk	hazard chain + RiskControl + residual eval
cybersecurity	threat model + STRIDE scenarios + controls
verification	VerificationCase · TestArtifact · Evidence
trace	all SemanticLink instances across layers
views, document_views	DiagramView + DocumentBackedView (RMF, SDD, V&V)
methodology	ProjectMethodBinding for GPCA

Software components and the requirements they satisfy

Real instances — not pseudo-code. SafetyClassKind::C is an enum; periodMs / WCET are typed; statements are full SysML.

examples/gpca/gpca_architecture.sysml

```
package md::examples::gpca::architecture {


part tlm : SoftwareComponent {
    attribute id              =
"SW-001";
    attribute name            =
"Top_Level_Mode";
    attribute responsibility  =
"Ensure ON/OFF, startup/shutdown checks";
    attribute safetyClass     =
SafetyClassKind::C;
    attribute periodMs        =
20.0;
    attribute deadlineMs      =
20.0;
    attribute wcetMs          =
2.0;
    attribute schedulingPolicy =
fixedPriorityPreemptive;
  }
```

examples/gpca/gpca_requirements.sysml

```
package md::examples::gpca::requirements {


requirement needSafeTherapy : StakeholderNeed {
    attribute id        =
"NEED-001";
    attribute statement =
"Support safe patient-controlled
                            analgesic infusion therapy."
;
    attribute priority  =
"high";
  }


requirement reqLockout : SoftwareRequirement {
    attribute id           =
"REQ-025";
    attribute statement    =
"Ignore patient bolus requests that
                            violate lockout or max bolus."
;
    attribute sourceKind   =
RequirementSourceKind::risk;
    attribute concernKind  =
```

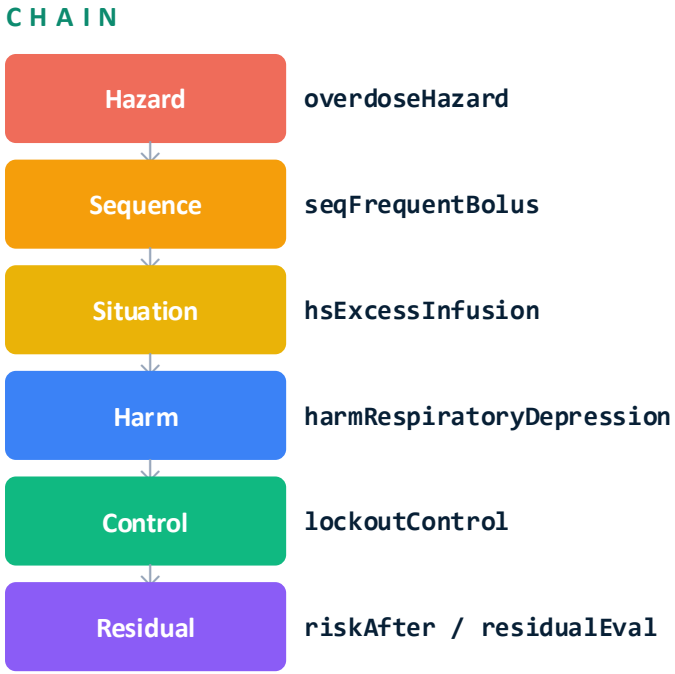
part alarm : SoftwareComponent { /* SW-003 · Class C */ }

Hazard → Sequence → Situation → Harm → Risk → Control → Residual

Six-link chain expressed as RiskTraceLink instances. Each node is typed; severity, probability, acceptability are enums.

examples/gpca/gpca_risk.sysml

```
package md::examples::gpca::risk {


part overdoseHazard : Hazard {
    attribute hazardType  =
HazardTypeKind::overdose;
    attribute foreseeable =
true;
  }


part seqFrequentBolus        : SequenceOfEvents;

part hsExcessInfusion        : HazardousSituation;

part harmRespiratoryDepression : Harm {
    attribute criticality =
CriticalityKind::catastrophic;
  }


part lockoutControl : RiskControl {
    attribute controlKind =
RiskControlKind::inherentSafeDesign;
  }
```



Traceability is data — not a matrix

Every link is a typed instance with status. The RMF view, V&V matrix, and impact analysis all read the same SemanticLink instances.

examples/gpca/gpca_trace.sysml

```
package md::examples::gpca::trace {


  // Need → Requirement

  part linkNeedToModeReq : RequirementSourceLink {
      attribute sourceRole       =
"stakeholder need";
      part      sourceDriver     = needSafeTherapy;
      part      targetRequirement= reqSingleMode;
    }


  // Risk → Software Requirement

  part linkRiskToLockoutReq : RequirementSourceLink {
      attribute sourceRole       =
"risk driver";
      part      sourceDriver     = riskBefore;
      part      targetRequirement= reqLockout;
    }

  // Requirement → Component
```

ONE CLOSED THREAD

Three horizons, one trajectory

Adoption-first roadmap. Earn trust, then grow scope.

NOW · 2026

H1

Prove the backbone

- open md:: package 1.0.0 (core, architecture, methodology, viewpoints, views, compliance)
- GPCA worked example as proof artifact
- SysML v2 source, CLI + viewer
- seed adoption with one design partner per modality

2026–2027

H2

Compile compliance

- auto-compiled DocumentBackedView (RMF, SDD, V&V) from one model
- typed-link impact analysis & stale-evidence detection
- extra layers: AI/ML, alarms, usability
- PLM & ALM export adapters

2027+

H3

Ecosystem & assurance

- device-specific methodology profiles
- auditor-readable assurance cases
- cross-domain bridges to ARP4761 / ISO 26262 patterns
- community-governed ontology evolution

Sequence. Unified model first, then compliance outputs, then ecosystem — never the other way around.

Adopt the semantic backbone.

Build safer medical devices. Prove it with confidence.

FASTER CHANGES

with lower risk

STRONGER CASES

with for regulators

BETTER QUALITY

with less rework

GET INVOLVED

md:: 1.0.0 · Open source · SysML v2 · ISO 42010 aligned · INCOSE Medical SE WG

MEMO